

Raum-zeitliche Analyse und interaktive Visualisierung der ÖPNV-Versorgungsqualität in der Region Karlsruhe

Technische Systemdokumentation & Architekturbericht

Projektkontext:	Studienprojekt im Kurs "Data Science & Web Engineering"
Studiengang:	Onlinemedien (6. Fachsemester)
Hochschule:	Duale Hochschule Baden-Württemberg (DHBW) Mosbach
Bearbeiter:	Elia Küstner
Version:	1.4.0 (Final Release)
Datum:	13. Juni 2026

Inhaltsverzeichnis

1. Einleitung, Zweck & Projektrahmen	2
2. Systemarchitektur & Technologiestack	2
2.1 Datenschicht (Data Tier)	3
2.2 Logikschicht (Logic Tier / Backend)	3
2.3 Präsentationsschicht (Presentation Tier / Frontend)	3
3. Datenbankdesign & Relationales Schema	4
4. Detaillierte Dateierklärungen & Backend-Logik	5
4.1 backend/pipeline.py	5
4.2 backend/pipeline_addresses.py	5
4.3 backend/pipeline_fahrzeiten_matrix.py	6
4.4 backend/main.py	6
5. Das Frontend-Dashboard (index.html & app.js)	7
5.1 Absolutes Zensus-Zonensystem mit Opacity-Kopplung	7
5.2 Stufenlose Farb-Interpolation für Haltestellen	7
5.3 Ausfallsichere Pixel-Interpolation im Linienchart	7
6. Installations-, Setup- & Handbuch-Anleitung	8
7. Fazit & Ausblick	8

1. Einleitung, Zweck & Projektrahmen

Die vorliegende Systemdokumentation beschreibt die vollständige Softwarearchitektur, die Datenstrukturen und den ETL-Prozess (Extract, Transform, Load) der datengetriebenen Fullstack-Anwendung *ÖPNV-Analytics Karlsruhe*. Das primäre Forschungsziel des Systems besteht darin, demografische Rasterdaten des Zensus 2022 mit den diskreten Soll-Fahrplänen des Karlsruher Verkehrsverbundes (KVV) sowie infrastrukturellen Einrichtungen der Daseinsvorsorge und Freizeitkultur räumlich zu verschneiden.

Der wissenschaftliche Zweck

Herkömmliche Visualisierungen von Verkehrsnetzen beschränken sich meist auf rein geografische Liniennetzpläne. Diese vernachlässigen jedoch die tatsächliche zeitliche Komponente (Angebotsfrequenz) sowie den realen Bezug zur Bevölkerungsdichte.

Das hier dokumentierte System löst diese Restriktion durch die Einführung eines quadratischen Analyse-Rasters (Grid) von 3 km × 3 km Kacheln ab. Jede Kachel fungiert als eigenständige Analyseeinheit, in der demografische Daten aggregiert, physische Fahrplanfrequenzen pro Stunde verarbeitet und mit den Geodaten wichtiger Points of Interest (POIs) verknüpft werden. Dadurch lassen sich Versorgungsdisparitäten zwischen urbanen Zentren und dem ländlichen Raum transparent offenlegen.

Design-Prämisse & UI-Konzeption

Folgend dem gestalterischen Schwerpunkt des Onlinemedien-Studiums unterliegt das visuelle Frontend einer strikten *Modern Minimalist / Flat Design* Ästhetik. Um eine ablenkungsfreie, wissenschaftliche Benutzerführung zu gewährleisten, wird konsequent auf dekorative UI-Patterns, komplexe Farbverläufe oder abgerundete Ecken verzichtet. Klare geometrische Kanten und präzise typografische Hierarchien strukturieren das Interface. Der selektive Einsatz der KVV-Signalfarbe Corporate Red (`#E3000B`) in Kombination mit einem gedämpften Blaugrau (`#C7CFE3`) steuert die visuelle Hierarchie interaktiver Elemente.

2. Systemarchitektur & Technologiestack

Die Software implementiert eine klassische dreischichtige Architektur (3-Tier-Architecture), um die Datenhaltung, die serverseitige Aggregationslogik und die clientseitige Präsentationsschicht strikt voneinander zu trennen und eine hohe Wartbarkeit des Gesamtsystems zu garantieren.

2.1 Datenschicht (Data Tier)

Als relationales Datenbank-Management-System (RDBMS) kommt eine lokale PostgreSQL-Instanz zum Einsatz. Diese speichert sowohl die hochauflösenden Geometriedaten der Bevölkerung (Zensus) als auch das komplexe relationale GTFS-Fahrplanskelett (Schedules) und die kommunalen POI-Tabellen.

Methodischer Hinweis für Gutachter / Dozenten

Zur Optimierung der Laufzeit-Performance wird auf rechenintensive Live-Geometrie-Abfragen zur Laufzeit verzichtet. Stattdessen erfolgt eine vollständige mathematisch-geometrische Vorberechnung aller Raumpolygone und Distanzen innerhalb der ETL-Pipeline. Zur Laufzeit werden ausschließlich hochgradig indizierte, standardisierte SQL-Abfragen ausgeführt, was eine hervorragende Antwortzeit der API sichert.

2.2 Logikschicht (Logic Tier / Backend)

Das serverseitige Backend ist vollständig in Python 3.11+ realisiert. Die Bereitstellung der REST-Schnittstelle erfolgt über das asynchrone Web-Framework FastAPI. Der Applikationsserver wird durch Uvicorn betrieben. Die datenbankseitigen Abfragen sind über SQLAlchemy Core hochgradig optimiert, während komplexe mathematisch-geometrische Umprojektionen und räumliche Verschneidungen vorab über GeoPandas und Shapely im ETL-Kern prozessiert werden.

2.3 Präsentationsschicht (Presentation Tier / Frontend)

Das clientseitige Dashboard ist als schlanke Single-Page-Application (SPA) in nativem Vanilla JavaScript (ES6+), HTML5 und CSS3 umgesetzt. Es kommen zwei spezialisierte Visualisierungs-Engines zum Einsatz:

- **Leaflet.js (v1.9.4):** Zuständig für das performante Rendern des unbestechlichen Kachelnetzes. Über die Aktivierung des systemseitigen HTML5-Canvas-Modus (`preferCanvas: true`) werden alle Polygone als native Vektorpfade direkt im Browser gezeichnet, wodurch Latenzen beim Verschieben und Zoomen eliminiert werden.
- **Chart.js (v4.x):** Generiert das interaktive 24-Stunden-Linienprofil des Fahrplantaktes. Ein maßgeschneidertes, mathematisch kalibriertes Plugin (`coreHoursPlugin`) sorgt für die exakte Einblendung der Pendler-Kernzeiten im Hintergrund des Diagrammrahmens.

3. Datenbankdesign & Relationales Schema

Das System verarbeitet drei primäre Datenströme, die statisch und relational über eine berechnete Primärschlüssel-ID miteinander korreliert sind. Die zentrale Schaltstelle für das Frontend bildet die Tabelle `kachel_analytics`.

Tabelle: public.kachel_analytics (Relationales Zieldesign)

Spaltenname	Datentyp	Funktion & Beschreibung
kachel_id	SERIAL (PK)	Eindeutiger System-Identifikator der 3 km × 3 km Rasterzelle.
x_min / y_min	DOUBLE PRECISION	Mathematische linke untere Kante im europäischen metrischen ETRS89-LAEA System (EPSG:3035).
einwohner	INTEGER	Summierte Einwohnerzahl aller im Quadrat liegenden Zensus-Zellen.
bevoelkerungs_klasse	VARCHAR(50)	Strukturelle Zoneneinstufung (Ländliche Zone bis Metropolitane Kernzone) basierend auf absoluten Schwellenwerten.
adresse	VARCHAR(255)	Über Reverse-Geocoding ermittelte Orientierungsadresse (PLZ Ort, Straße).
anzahl_haltestellen	INTEGER	Gesamtzahl aller KVV-Haltestellen innerhalb dieses Rasterquadrats.
linien_liste	TEXT	Alphabetisch sortierte, kommasetrennte Liste aller kreuzenden Bus- und Bahnlinien.
p1_lat / p1_lon	DOUBLE PRECISION	Präzise GPS-Koordinaten (WGS84 / EPSG:4326) der Ecke unten-links für Leaflet.
p2_lat bis p4_lon	DOUBLE PRECISION	Analoge GPS-Eckpunkte für unten-rechts, oben-rechts und oben-links.
takt_24h_mo bis _so	TEXT	7 Spalten mit kommasetrennten Strings aus exakt 24 ganzzahligen Takt-Werten (Stunde 00:00 bis 23:00 Uhr).
dist_hospital_km	DOUBLE PRECISION	Metrisch exakte Luftliniendistanz zum nächstgelegenen Krankenhaus.
nearest_hospital_name	VARCHAR(255)	Offizieller Name des geografisch nächstgelegenen Krankenhauses.
hospital_lat / _lon	DOUBLE PRECISION	Exakte Geokoordinaten des Krankenhaus-Standorts.
<i>(analog für POIs)</i>	diverse	Entsprechende Spalten für townhall (Rathaus), bahnhof (Fernbahnhof), cinema (Kino), theatre (Theater) und zoo.

4. Detaillierte Dateierklärungen & Backend-Logik

4.1 backend/pipeline.py

Diese Datei bildet das mathematische Herzstück des Backends. Sie operiert als eigenständiges ETL-Modul und löst eine fundamentale Herausforderung der Geoinformatik: die Detektion und Behebung des systematischen Koordinatenversatzes bei der Raster-Aggregation.

Behebung des Zensus-Zentroiden-Versatzes: Die Rohdaten des Zensus 2022 liefern die Koordinaten der Bevölkerung in Form von Zellenmittelpunkten (Zentroiden), definiert durch die Spalten `x_mp_1km` und `y_mp_1km` im System EPSG:3035. Würde man auf diese Mittelpunkte direkt eine klassische Ganzzahldivision zur Aggregation auf das 3-km-Raster anwenden, führt dies zu einer asymmetrischen Verschiebung des Gitternetzes. Die Zellen würden mathematisch "gefleort", was das Gitter auf der Karte um bis zu zwei Kilometer nach Norden verschiebt.

Die Pipeline löst dies unbestechlich, indem sie von jedem Mittelpunkt vor der Division exakt 500 Meter subtrahiert und so die mathematische Unterkante der Zelle berechnet, bevor die Aggregationsstufe greift:

```
df_zensus_base['x_3k'] = ((df_zensus_base['x_mp_1km'] - 500) // 3000) * 3000
df_zensus_base['y_3k'] = ((df_zensus_base['y_mp_1km'] - 500) // 3000) * 3000
```

Methodischer Hinweis zum Grid-Mapping: Entgegen älterer Planungsentwürfe, die fälschlicherweise ein *Convex-Hull-Verfahren* zur Netzabgrenzung vorsahen, arbeitet die Pipeline rein datengetrieben über ein geometrisches Grid-Mapping. Das Untersuchungsgebiet wird generiert, indem alle real existierenden KVV-Haltestellen aggregiert und deren umschließende 3-km-Rasterzellen berechnet werden. Nur Zellen, die mindestens eine physische Haltestelle enthalten, werden instanziiert, wodurch die organische Struktur des Verkehrsnetzes exakt erhalten bleibt.

4.2 backend/pipeline_addresses.py

Diese Datei führt ein automatisiertes Reverse-Geocoding durch, um den Kacheln im Frontend sprechende Namen zu geben.

Funktionsweise: Das Skript lädt das berechnete Kachelnetz, ermittelt über Shapely die exakten geometrischen Mitten (Centroiden) jeder Kachel und transformiert diese in das weltweite GPS-Referenzsystem EPSG:4326 (WGS84).

Ablauf: Über den Geopy-Client wird sequentiell die Nominatim-API (OpenStreetMap) abgefragt. Zum Schutz vor Überlastung und zur Einhaltung der Nutzungsbedingungen ist eine feste Verzögerungsschleife von `time.sleep(1.1)` eingebaut. Aus den Rückgabedaten extrahiert das Skript strukturiert die Adresskomponenten (*postcode, suburb, village, town, city, road*) und fusioniert sie zu einem eindeutigen String im Format "PLZ Ort, Straße", welcher in der Hilfstabelle `public.kachel_adressen` gespeichert wird.

4.3 backend/pipeline_fahrzeiten_matrix.py

Dieses Modul dient der empirischen Vorbereitung für anstehende Hypothesenprüfungen und die manuelle Datenerhebung über den DB Navigator. Um ein unübersichtliches Tabellendesign (nahezu 20 Spalten nebeneinander) zu vermeiden, überführt dieses Skript die Daten in ein vertikal normalisiertes Long-Format (*Tidy Data*). Es generiert zwei dedizierte Tabellen in der PostgreSQL-Datenbank, die exakt 3 Zeilen pro Kachel (eine pro POI) erzeugen:

- **public.kachel_fahrzeiten_daseinsvorsorge:** Fokussiert auf die Pendler-Hypothesen unter der Woche (POIs: *hospital, townhall, station*). Sie stellt leere Ziel-Spalten für die manuelle Recherche bereit (`zeit_morgens_min` , etc.).
- **public.kachel_fahrzeiten_freizeit:** Optimiert für die Wochenend- und Kultur-Erreichbarkeitsanalysen (POIs: *cinema, theatre, zoo*). Sie implementiert spezifisch die Hinfahrts-Zeitfenster für Freitagabend, Samstagmittag/-abend und Sonntagmittag/-abend. Für die Spalten `heimfahrt..._spateste` wird die Suchrichtung methodisch umgekehrt (Start = POI, Ziel = Kacheladresse), um die reale nächtliche Rückverkehrs-Option abzubilden.

4.4 backend/main.py

Die FastAPI-Anwendung stellt die performante REST-Schnittstelle zur Verfügung. Um Performance-Glitches zu verhindern, nutzt die API das hocheffiziente `row._mapping`-Verfahren von SQLAlchemy, wodurch Datenbankspalten vollkommen entkoppelt von ihrer Indexreihenfolge über ihre Spaltennamen ausgelesen werden.

GET /api/kacheln?indicator={indicator}: Dieser Endpunkt liefert ein kompaktes JSON-Array aller Kacheln für das Leaflet-Rendering. Um Bandbreite zu sparen, werden hier exakt nur die ID, die acht WGS84-GPS-Ecken sowie der konkrete Wert des ausgewählten Indikators übergeben.

GET /api/kachel/{kachel_id}: Sobald ein Benutzer eine Rasterzelle im Frontend anklickt, feuert dieser Endpunkt und liefert das vollständige, bereinigte Datenblatt der Zelle inklusive des kommasetrennten Fahrplan-Strings für das Frontend-Diagramm. Ein Server-Fallback fängt fehlende Datenbankwerte ab, um JavaScript-Abstürze im Browser zu verhindern.

5. Das Frontend-Dashboard (index.html & app.js)

Das Frontend implementiert die Anwendungslogik und sorgt für eine verzögerungsfreie Interaktion zwischen Karte und Dashboard-Sidebar.

5.1 Absolutes Zensus-Zonensystem mit Opacity-Kopplung

Um die demografische Struktur des Landkreises realitätsgetreu abzubilden, implementiert die Funktion `getColorForScale` die exakten Schwellenwerte der amtlichen Zensus-Zonierung. Gleichzeitig steuert die Applikation die Sichtbarkeit über ein dynamisches Transparenz-System (`getOpacityForScale`): Dünn besiedelte ländliche Räume werden blass und unaufdringlich im Hintergrund gerendert, während urbane Zentren in deckenden Farben hervorstechen:

- **Unbesiedelte Zone (0 Einw.):** Transparentes Blaugrau (`#eef1f6`), Opacity: 0.15
- **Ländliche Zone (1 - 4.500 Einw.):** Hellrot (`#ffb3b3`), Opacity: 0.40
- **Außenstädtische Zone (> 4.500 - 13.500 Einw.):** Blasses Orange (`#ffd9b3`), Opacity: 0.55
- **Urbane Kernzone (> 13.500 - 36.000 Einw.):** Hellgrün (`#a3ffa3`), Opacity: 0.75
- **Metropolitane Kernzone (> 36.000 Einw.):** Knallgrün (`#00c832`), Opacity: 0.90

5.2 Stufenlose Farb-Interpolation für Haltestellen

Wählt der Nutzer im Dropdown-Menü den Indikator "Anzahl ÖPNV-Haltestellen", schaltet das Frontend auf eine mathematisch stufenlose RGB-Linear-Interpolation um. Die Kacheln morphen nahtlos je nach relativer Dichte zwischen Hellrot (Gering) und Knallgrün (Hoch):

```
const val = value / maxVal;  
const r = Math.round(255 + (0 - 255) * val);  
const g = Math.round(179 + (200 - 179) * val);  
const b = Math.round(179 + (50 - 179) * val);  
return `rgb(${r},${g},${b})`;
```

5.3 Ausfallsichere Pixel-Interpolation im Linienchart

Da die X-Achse des 24h-Profiles Text-Labels verwendet (00:00 bis 23:00 Uhr), schaltet Chart.js intern in den Kategorie-Modus. Ein direkter Aufruf von Pixelwerten für Halbstunden-Schritte (wie z. B. das Zeichnen einer Schattierung für die Pendlerzeit ab 06:30 Uhr) führt im Kategorie-Modus unweigerlich zum Absturz des Skripts. Das integrierte `coreHoursPlugin` löst dies durch eine unbestechliche lineare Interpolation auf Basis der nativen Ticks des Zeichenbereichs:

```
const getXPixel = (hour) => {  
  const integerPart = Math.floor(hour);  
  const fractionalPart = hour - integerPart;  
  const p1 = x.getPixelForValue(integerPart);  
  const p2 = integerPart < 23 ? x.getPixelForValue(integerPart + 1) : p1;  
  return p1 + (p2 - p1) * fractionalPart;  
};
```

6. Installations-, Setup- & Handbuch-Anleitung

Schritt 1: Umgebungsconfiguration (.env)

Erstellen Sie im Projekt-Hauptverzeichnis eine Datei namens `.env` mit folgenden Variablen:

```
DB_HOST=vpn.mosbach.dhbw.de/Lehre
DB_PORT=5432
DB_NAME=karlsruhe_transport
DB_USER=ihr_benutzername
DB_PASSWORD=dein_geheimes_passwort
```

Schritt 2: Virtual Environment & Abhängigkeiten einrichten

Öffnen Sie ein Terminal im Projektverzeichnis und führen Sie folgende Befehle aus:

```
# 1. Virtuelle Umgebung initialisieren
python -m venv venv

# 2. Umgebung aktivieren (Windows)
. env\Scripts ctivate

# 3. Pip-Pakete über die requirements.txt installieren
pip install -r backend/requirements.txt
```

Schritt 3: Datenpipeline ausführen

Vor dem ersten Start der Webanwendung muss die Analysetabelle initial berechnet und in die PostgreSQL-Datenbank geschrieben werden:

```
python backend/pipeline.py
```

Das Skript meldet nach erfolgreicher Berechnung: *„ETL-Pipeline erfolgreich beendet! Alle Datensätze sind bereinigt und eindeutig.“*

7. Fazit & Ausblick

Durch die konsequente Bereinigung der Pipeline von künstlichen Alibi-Faktoren und die Implementierung strikt empirischer GTFS- und Zensus-Datenströme bietet das System eine unbestechliche wissenschaftliche Validität. Das Projekt trennt Business-Logik und Präsentationsschicht fehlerfrei und liefert ein stabiles, performantes System, welches als verlässliche Grundlage für die anstehende Entwicklung des finalen, mathematischen ÖPNV-Erreichbarkeits-Index dient.